PROGRAM-5

Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB

Dataset (users collection) inserted into MongoDB:

```
db.users.insertMany([
  { name: "Alice", age: 30, gender: "Female", city: "New York" },
  { name: "Bob", age: 25, gender: "Male", city: "Los Angeles" },
  { name: "Charlie", age: 35, gender: "Male", city: "Chicago" },
  { name: "David", age: 28, gender: "Male", city: "San Francisco" },
  { name: "Eve", age: 22, gender: "Female", city: "Boston" },
  { name: "Frank", age: 40, gender: "Male", city: "Houston" },
  { name: "Grace", age: 27, gender: "Female", city: "Miami" },
  { name: "Hannah", age: 29, gender: "Female", city: "Seattle" },
  { name: "Isaac", age: 31, gender: "Male", city: "Denver" },
  { name: "Jack", age: 26, gender: "Male", city: "Dallas" }
])
```

1. Count total documents

```
db.users.countDocuments({})
```

Output:

```
10
```

2. Count users older than 25

```
db.users.countDocuments({ age: { $gt: 25 } })
```

Output:

```
7
```

3. Sort users by age (ascending)

```
db.users.find().sort({ age: 1 })
```

Output:

```
[
  { "name": "Eve", "age": 22, "gender": "Female", "city": "Boston" },
  { "name": "Bob", "age": 25, "gender": "Male", "city": "Los Angeles" },
  { "name": "Jack", "age": 26, "gender": "Male", "city": "Dallas" },
  { "name": "Grace", "age": 27, "gender": "Female", "city": "Miami" },
  { "name": "David", "age": 28, "gender": "Male", "city": "San Francisco" },
  { "name": "Hannah", "age": 29, "gender": "Female", "city": "Seattle" },
  { "name": "Alice", "age": 30, "gender": "Female", "city": "New York" },
  { "name": "Isaac", "age": 31, "gender": "Male", "city": "Denver" },
```

```
  { "name": "Charlie", "age": 35, "gender": "Male", "city": "Chicago" },
  { "name": "Frank", "age": 40, "gender": "Male", "city": "Houston" }
]
```

4. Skip Documents

Skips a certain number of documents.

```
db.users.find().skip(9)
```

Output:

5. Aggregation Pipeline

Aggregation is used for data processing and transformation.
Example: Count users older than 25, group by gender, and sort by count.

```
db.users.aggregate([
  { $match: { age: { $gt: 25 } } }, // Filter users older than 25
  { $group: { _id: "$gender", count: { $sum: 1 } } }, // Group by gender
  { $sort: { count: -1 } }, // Sort in descending order
  { $limit: 5 } // Limit to top 5 results
])
```

Combining Operations

Example: Get 5 users sorted by age in descending order, skipping the first 3.

```
db.users.find().sort({ age: -1 }).skip(3).limit(5)
```